



- ▶ **Estructuras condicionales:** if-else-elif
- ▶ **Estructuras iterativas:** while, for-in
- ▶ **Manejo de excepciones:** try-except

Bibliografía

- *Introducción a la programación con Python. Capítulo 4*, Andrés Marzal, Isabel Gracia y Pedro García.
- *Computational Physics, cap. 2*, Mark Newman.
- *Programming for computations- Python, secciones: 3.3 y 3.4*, Evein Linge and Petter Langtangen.

Estructuras de control

Los programas que hemos visto son lineales: piden información, ejecutan ciertas órdenes y muestran una salida. Sin embargo, ¿qué pasaría si en `primer_programa.py` se ingresa una edad de -3. Claramente, no tiene sentido físico y, aún así, el programa seguirá corriendo entregando resultados erróneos. Peor aún, si la persona ingresa una letra en la edad, el programa colapsa. Una característica que todo programa en Física computacional debe tener es la habilidad de romper esa linealidad: decidir qué líneas ejecutar y cuáles saltarse, o tomar decisiones sobre la marcha basándose en un criterio previo. De esto se encargan las estructuras de control.

Existen dos tipos fundamentales de estructuras de control:

- **Estructuras condicionales:** `if`, `elif` y `else`.
- **Estructuras iterativas (o bucles):** `while` y `for`.

Los condicionales if-elif-else

La estructura de un condicional en Python consta de la palabra clave `if-else`, seguida de la condición a evaluar y de dos puntos (`:`). Los dos puntos indican al intérprete que a continuación viene el bloque de instrucciones que deben ejecutarse únicamente si la condición es verdadera. La sintaxis es la siguiente:

```
1 if condición:
2     # Bloque de código si la condición se cumple
3 else:
4     # Bloque de código si la condición NO se cumple
```

if: «al llegar a aquí evalúa estas acciones solo si esta condición es cierta». **else:** «en caso contrario, entonces evalúa esta otra acción »

En la mayoría de los lenguajes de programación (como C++ o Fortran), se utilizan llaves {} o palabras reservadas para delimitar dónde empieza y termina una condición. En Python no es así: los espacios al inicio de la línea (sangría o *indentation*) definen la estructura del programa.

■ ¿Cómo sabe Python qué está dentro del if?

Todas las líneas que estén desplazadas hacia la derecha pertenecen al bloque del **if** (o del **else**). En cuanto escribimos una línea pegada al margen izquierdo, esa instrucción queda por fuera del condicional y se ejecutará de forma lineal, sin importar si la condición fue verdadera o falsa.

■ Espacios vs. Tabuladores:

Aunque Python acepta cualquier cantidad de espacios para definir un bloque, la regla estándar en la comunidad (PEP 8) es utilizar exactamente 4 espacios por cada nivel de sangría. La mayoría de los editores como VS Code insertan estos 4 espacios automáticamente al presionar Enter después de los dos puntos (:).

Volviendo a nuestro ejemplo de la edad, podemos reorganizar la lógica usando un bloque **if-else** para que el cálculo solo se realice con edades válidas:

primer_programa_v2.py

```
1 """ Nuestro primer programa en Python.
2 Modificado con el condicional if"""
3
4 # con el comando print(""), imprimimos un mensaje en cosa \n salta la linea
5 print("¡Hola, espero que te encuentres muy bien!")
6
7 #El comando input se utiliza para ingresar información.
8 #Por defecto la variable asignada es un string
9 nombre = input("¿Cuál es tu nombre?\n")
10
11 print(f"-----")
12 # Uso de f-strings para la salida
13 print(f"¡Mucho gusto, {nombre}!")
14
15 # Conversión de datos.
16 # int() convierte la entrada a un entero
17 print(f"-----")
18 edad= int(input("¿Cuántos años tienes?\n"))
19 if edad < 0:
20     print("Tu edad debe ser positiva, suerte llave")
21 else:
22     print(f"-----")
23     # Hacemos operaciones con ese entero
24     edad_segundos = edad * 365 * 24 * 3600 #calcula edad en segundos
25     edad_minutos = edad * 365 * 24 * 60 #calcula edad en minutos
26     futura_edad = edad + 5 # edad dentro de 5 años
27     print(f"Tu edad en minutos es {edad_minutos}. En notación científica es
{edad_minutos:.2e} minutos")
```

```

28     print(f"Tu edad en segundos es {edad_segundos}. En notación científica
29     es {edad_segundos:.2e} segundos")
30     print(f"Si todo sale bien, te graduarías de Física a los {futura_edad}
    años.")
31     print("-----")

```

Diferentes condiciones pueden ser usadas en el `if` por ejemplo, $x == a$ compara la igualdad y $x != a$ compara la diferencia. Note el uso del doble signo igual.

Cuando un problema requiere evaluar más de dos alternativas mutuamente excluyentes, la combinación básica de `if` y `else` se queda corta. Para evitar anidar múltiples bloques (lo que vuelve el código más largo e ilegible), Python ofrece la instrucción `elif` (abreviación de *else if*).

La estructura `elif` nos permite encadenar tantas condiciones como sean necesarias. El programa evaluará cada condición de arriba hacia abajo de forma secuencial: en cuanto una de ellas resulte verdadera, ejecutará su bloque de código y saltará inmediatamente todo el resto del condicional. Consideremos el siguiente ejemplo (cabe resaltar que el mismo código puede ser escrito sin usar `elif`, solo anidando varios `if-else`),

piscina.py

```

1  """ Programa que ilustra el uso de los condicionales """
2  T = float(input("¿Cómo está la temperatura del agua?\n Entregame el valor en
    Celsius\n"))
3  if T > 20 and T < 27:
4      print('te puedes meter a la piscina')
5  elif 15 <= T <= 20:
6      print('El agua está medio fría, usted verá')
7  else:
8      print('No se meta a la piscina')

```

Si se ingresa por ejemplo una temperatura de 22,3 °C, la primera condición del `if` se evalúa como verdadera, el programa imprime su mensaje y se salta todo el resto del bloque. Para temperaturas intermedias, como $T = 16$ °C, la primera condición resulta falsa, lo que le indica al intérprete que debe avanzar hacia la línea del `elif`. En este punto la segunda condición resulta verdadera, por lo que imprime su mensaje y omite el resto del código, sin llegar a revisar el `else`. Si ninguna de las condiciones anteriores se cumple, como en temperaturas menores a 15 °C, el programa cae directamente en el `else` final. Si quisiéramos refinar aún más nuestras recomendaciones para clasificar el agua en más rangos, bastaría con añadir más cláusulas `elif` intermedias.

Ejemplo: Para terminar el estudio de los condicionales, considere el siguiente ejemplo

menu.py

```

1  """ Programa que ilustra el uso de los condicionales. Pide el radio
2  y calcula el diámetro, perímetro y área de un círculo """
3  from math import pi
4
5  radio = float(input('Entrégame el radio de un círculo en metros: '))
6
7  print('#####')
8  print('Escoge una opción del siguiente menú:')

```

```
9 print('a) Calcular el diámetro del círculo.')
10 print('b) Calcular el perímetro del círculo.')
11 print('c) Calcular el área del círculo.')
12 option= input('Escoge a, b o c y pulsa enter: ')
13
14 if option == 'a': # Cálculo del diámetro.
15     diametro = 2 * radio
16     print(f'El diámetro es {diametro:.2f} metros')
17 elif option == 'b': # Cálculo del perímetro.
18     perimetro = 2 * pi * radio
19     print(f'El perímetro es {perimetro:.2f} metros')
20 elif option == 'c': # Cálculo del área.
21     area = pi * radio ** 2
22     print(f'El área es {area:.2f} metros cuadrados')
23 else:
24     print(f'Solo hay tres opciones: a, b o c.')
```

Problemas:

1. Escriba un programa que lea un número entero y que muestre si es par o impar. Sugerencia: use la operación módulo y recuerde que un número es par si al dividirlo por 2 su residuo es cero y es impar en caso contrario.
2. Escriba un programa que lea los números reales a, b, c y entregue la solución de la ecuación cuadrática $ax^2 + bx + c = 0$.
3. Escriba un programa que solicite un número real. y que imprima si el número es estrictamente positivo, estrictamente negativo o si es exactamente cero.
4. Escriba un programa equivalente a piscina.py, pero solo usando **if-else**.
5. A presión atmosférica estándar (1 atm), el agua cambia de fase según su temperatura en grados Celsius (T):
 - $T \leq 0$ °C: Estado sólido (hielo).
 - $0 < T < 100$ °C: Estado líquido.
 - $T \geq 100$ °C: Estado gaseoso (vapor).

Cree un programa que reciba la temperatura y despliegue la fase correspondiente.

6. La luz visible se clasifica según su longitud de onda (λ) en nanómetros (nm). Consulte el espectro electromagnético y escriba un programa que lea λ y determine el color aproximado dentro del rango visible (380 nm a 750 nm). Si la longitud de onda está fuera de este rango, debe indicar si pertenece al ultravioleta o al infrarrojo.

La sentencia while

En inglés «while» significa «mientras». Lo que convierte a esta estructura de control en una iterativa, que se ejecuta mientras alguna condición sea cierta. Su sintaxis y apariencia son muy similares a las del

`if`, pero su comportamiento introduce una repetición controlada. La instrucción `while` se usa así:

```
1 while condición:
2     acción
3     acción
```

Significa que «mientras se cumpla esta condición, repita estas acciones».

piscina_while.py

```
1 """ Programa que ilustra el uso del while """
2 T = float(input("¿Cómo está temperatura del agua?\n Entregame el valor en
   Celsius\n"))
3 while T <= 15 or T >= 27:
4     print('No se meta a la piscina')
5     break
6 print('Te puedes meter a la piscina')
```

Al igual que ocurre con el `if`, la instrucción `while` evalúa si la condición dada se cumple (en este caso, si $T \leq 15^\circ\text{C}$ o $T \geq 27^\circ\text{C}$). Si la condición es verdadera, ejecuta inmediatamente el bloque de código indentado que le sigue. En este caso, avisa que no se meta a la piscina y luego se rompe el ciclo con la instrucción `break`; de lo contrario, se salta dicho bloque.

Note que si el programa muestra que no ingrese a la piscina, entonces ya no puede ingresar otra temperatura, se rompe. Esto puede cambiar si agregamos `continue`. Podemos reescribir el programa de la piscina utilizando un bucle interactivo `while True` combinado con `continue` y `break`. En este esquema, el bucle solicita continuamente la temperatura hasta que se ingrese un valor dentro del rango adecuado.

piscina_while_v2.py

```
1 """ Programa que ilustra el uso del while, break y continue """
2 while True:
3     T = float(input("¿Cómo está la temperatura del agua?\n Entrégame el valor en
   Celsius:\n"))
4
5     if T <= 15 or T >= 27:
6         print('No se meta a la piscina.\n')
7         continue
8
9     print('Te puedes meter a la piscina.')
10    break
```

Para entender esta línea, debemos recordar que la instrucción `while` evalúa siempre una expresión que dé como resultado un valor booleano: verdadero (`True`) o falso (`False`).

Al escribir directamente `while True:`, estamos pasando la constante `True` como condición. Dado que el valor `True` nunca cambia, la condición siempre evalúa como verdadera. Esto crea lo que en programación se conoce como un bucle infinito explícito o bucle de ejecución continua.

En términos sencillos, le estamos diciendo al intérprete: «Ejecuta este bloque de código indefinidamente, repítelo una y otra vez sin parar». **Si la temperatura es inválida** ($T \leq 15$ o $T \geq 27$): El condicional resulta verdadero, imprime el mensaje de advertencia y ejecuta el `continue`. Esto hace que Python ignore por completo la instrucción `print` de éxito y el `break`, regresando inmediatamente a solicitar una nueva temperatura en la primera línea del `while`. **Si la temperatura es válida** ($15 < T < 27$):

El condicional resulta falso y el bloque del `if` se omite. El programa continúa hacia la confirmación de ingreso a la piscina y finaliza el ciclo ejecutando el `break`.

La diferencia entre `break` y `continue` es que **break**: Sale del bucle completamente. El programa continúa ejecutando las líneas de código que están fuera del bloque. En su lugar, **continue**: Omite las líneas de código restantes dentro del bloque en esa iteración específica y regresa al encabezado del bucle para procesar el siguiente elemento o iteración.

```

1 In [1]: x = 10
2
3 In [2]: while x > 0:
4     ...:     x = x-2
5     ...:     if x == 4:
6     ...:         continue
7     ...:     print(x)
8     ...:
9 8
10 6
11 2
12 0

```

```

1 In [15]: y = 5
2
3 In [16]: while y > 0:
4     ...:     print(y)
5     ...:     y = y -1
6     ...:     if y==3:
7     ...:         break
8     ...:     else:
9     ...:         print("hecho")
10    ...:
11 5
12 hecho
13 4

```

Pasemos a un segundo ejemplo (famosos) con el `while`. *La suma de Gauss*. Cuenta la historia que un profesor para mantener ocupados a sus estudiantes, les solicitó sumar los números enteros desde 1 hasta 100. Es decir, $1 + 2 + 3 \dots 100$. Puede leer acerca de esta anécdota en el siguiente [enlace](#). Gauss presentó su célebre expresión

$$\sum_{i=1}^n k = 1 + 2 + 3 \dots + n = \frac{n(n+1)}{2} \quad (1)$$

Note que para $n = 100$, la suma da $50 \cdot 101 = 5050$. Vamos a hacer lo mismo pero con el uso del `while`.

gauss.py

```

1 """ En este programa se comprueba la suma de Gauss"""
2 print("comprobación de la fórmula de Gauss:\n")
3 print("1+2+3+...+100 =5050\n")
4
5 k = 1      #iniciamos contador

```

```
6 suma = 0 #iniciamos el acumulador de la suma
7
8 while k <= 100:
9     suma = suma + k # recuerde que puede usar suma += k
10    k += 1 # k = k+1
11
12 print(f"El valor de la suma es: {suma} ")
```

Este programa se ejecuta de la siguiente manera:

- la variable suma inicia en 0 y el contador k inicia en 1.
- En la primera iteración, la condición $1 \leq 100$ es verdadera, entonces se suma 1 al acumulador ($\text{suma} = 1$) y se incrementa el contador ($k = 2$).
- El ciclo se repite actualizando las variables: ($1 + 2 = 3$), luego ($3 + 3 = 6$).....
- Cuando k alcanza el valor de 101, la condición $101 \leq 100$ resulta falsa, el bucle se rompe y la ejecución continúa fuera del bloque.

Para que quede más claro, acá se presenta el pseudocódigo

Pseudocódigo:

- 1) Inicio
- 2) $k = 1$
- 3) $S = 0$
- 4) Mientras $k \leq 100$ haga:
 - a) $S = S + k$
 - b) $k = k + 1$
- 5) Muestre S
- 6) Terminar

Problemas:

1. Modifique el programa gauss.py para que pida el número entero n y entregue el resultado de la suma. Compare en la salida su resultado con la fórmula de Gauss.
2. A lo largo de la historia, grandes mentes han desarrollado diferentes esquemas computacionales para calcular el número π . En este ejercicio consideraremos dos de estos esquemas: uno desarrollado por Leibniz (1646–1716) y otro por Euler (1707–1783).

El esquema de Leibniz se puede escribir como:

$$\pi = 8 \sum_{k=0}^{\infty} \frac{1}{(4k+1)(4k+3)}$$

mientras que una de las formas del esquema de Euler se expresa como:

$$\pi = \sqrt{6 \sum_{k=1}^{\infty} \frac{1}{k^2}}$$

Si solo se utilizan los primeros N términos de cada sumatoria como una aproximación a π , cada esquema modificado calculará π con un cierto error.

Escriba un programa que reciba N como entrada, calcule π para cada esquema y muestre los resultados y los compare con el valor de π que viene por defecto en la librería `math` de Python.

3. La cantidad de una sustancia radiactiva $N(t)$ después de t años se modela mediante $N(t) = N_0 e^{-\lambda t}$, donde N_0 es la masa inicial y λ es la constante de decaimiento. Escriba un programa que reciba N_0 y λ , y utilice un bucle `while` para calcular año a año ($t = 1, 2, 3 \dots$) el valor de $N(t)$, deteniéndose cuando la sustancia se reduzca a menos del 10 % de su masa inicial ($N(t) < 0,1N_0$).
4. Un vector en un espacio tridimensional es una tripleta de valores reales (x, y, z) . Construya un programa que permita operar con dos vectores. El usuario verá en pantalla un menú con las siguientes opciones:
 - 1) Introducir el primer vector
 - 2) Introducir el segundo vector
 - 3) Calcular la suma
 - 4) Calcular la diferencia
 - 5) Calcular el producto escalar
 - 6) Calcular el producto vectorial
 - 7) Calcular el ángulo (en grados) entre ellos
 - 8) Calcular la longitud de cada vector
 - 9) Finalizar

El bucle `for-in`

Existe otro tipo de bucle, el bucle `for-in` que se lee como «para todo elemento en ..., haga ...». A diferencia del bucle `while`, se emplea idealmente cuando conocemos de antemano el número de repeticiones a realizar o cuando recorremos colecciones de datos finitas. Un bucle `for-in` se representa así:

```
1 for i in serie de valores:
2     acción
3     acción
```

Para iterar podemos usar listas o podemos usar la herramienta fundamental para acompañar este bucle es la función `range(inicio, fin, paso)`, la cual genera una secuencia discreta de valores enteros en el intervalo $[inicio, fin)$. Note que no incluye al último valor y solo acepta números enteros. Veamos un

ejemplo simple.

saludos.py

```
1 """ ejemplo del uso del for"""
2 lista = ['David', 'Felipe', 'Jorge', 'Jennifer', 'Santiago', 'Saray']
3
4 for i in lista: # podemos usar listas
5     print(f'Hola {i} ')
6 print('-----')
7 for i in range(1,5,1): # también podemos usar range
8     print(i)
9 print('-----')
10 for i in range(1,50,2): # también podemos usar range
11     print(i)
```

Podemos calcular la suma de Gauss usando el for:

gauss_v2.py

```
1 """ En este programa se comprueba la suma de Gauss usando el for"""
2 print("comprobación de la fórmula de Gauss:\n")
3 print("1+2+3+...+100 =5050\n")
4
5 suma = 0 #iniciamos el acumulador de la suma
6
7 for i in range(1,101,1): #note que como el último número es 100, ponemos 101
8     suma = suma + i # recuerde que puede usar suma += i
9
10 print(f"El valor de la suma es: {suma} ")
```

Ejemplo: Tiempo de vuelo de una bola. Un problema típico en mecánica básica es el de lanzar una bola hacia arriba con cierta rapidez inicial v_0 desde una altura y_0 . Despreciando al medio en el cual se mueve la bola y asumiendo el modelo de partícula, el cambio de la altura con respecto al tiempo para la bola está dado por $y(t) = y_0 + v_0 t - \frac{1}{2} g t^2$. Note que existe un tiempo $T = t$ en el momento que llega al piso, que satisface $y(T) = \Rightarrow \frac{1}{2} g T^2 - v_0 T - y_0 = 0$ cuya solución es

$$T_{1,2} = \frac{v_0 \pm \sqrt{v_0^2 + 2gy_0}}{g}$$

Claramente la solución con signo negativo no tiene sentido físico.

tiempo_vuelo.py

```
1 """
2 Programa que calcula en tiempo de vuelo de un partícula que es
3 lanzada verticalmente hacia arriba desde una altura unicial y_0 y con
4 rapidedz v_0.
5
6 El tiempo calculado se compara con el tiempo exacto para estudiar
7 el error de presición de la máquina.
```

```
8 """
9 import math
10
11 print('-----')
12 y0 = float(input('Entre la altura de lanzamiento en metros\t'))
13 v0 = float(input('Entre la rapidez con la que lo lanzas en m/s\t'))
14 N = int(input("Entre el número de puntos para el paso del tiempo\t"))
15 g = 9.8 # aceleración gravitacional de la tierra en m/s^2
16 print('-----')
17
18
19 #----- solucion exacta -----
20 T = (v0 + math.sqrt( v0**2 + 2*g*y0) )/g
21 #-----
22
23 t_i = 0.0 # tiempo inicial
24 t_f = 10. # tiempo final
25 dt = (t_f - t_i) / (N -1)
26
27 t = [] #Lista vacía para el tiempo
28 y = [] #lista vacía para la altura
29
30 #con el for construimos los valores de la altura para cada tiempo
31 for i in range(N):
32     ti = t_i + i * dt #voy incrementando el tiempo
33     yi = y0 + v0 * ti - 1/2. * g * ti**2
34     t.append(ti) # el .append va llenando la lista vacía
35     y.append(yi)
36
37 # con el while buscamos que esa altura siempre sea positiva.
38 k = 0
39 while k < len(y) and y[k]>=0:
40     k = k +1
41
42 t_num=t[k]
43
44 # calculo de errores
45
46 err_abs = abs(t_num - T)
47 err_rel = (err_abs / T) * 100
48
49 print(f"El tiempo que queda la bola en el aire: {t_num:.4f} segundos")
50 print(f"El tiempo exacto en el que cae la bola es : {T:.4f} segundos")
51 print("\n--- ANÁLISIS DE ERROR Y PRECISIÓN ---")
52 print(f"Paso de discretización (dt) : {dt:.6f} s")
53 print(f"Error absoluto (|t_num - T|) : {err_abs:.6f} s")
54 print(f"Error relativo (%) : {err_rel:.4f} %")
55 print(f"La altura para ese tiempo es: {y[k]:.4e} metros")
```

Al ejecutar el programa notamos que al aumentar el número de puntos N . El error disminuye. La pregunta

es, ¿podemos hacer el N muy grande para que el error sea cero? La respuesta es no. Al hacer un dt muy pequeño y luego sumarlo con el t_i se debe tener en cuenta la precisión de la máquina. Es decir, suponga que va a sumar $10 + 10^{-15} \sim 10$. Esto se conoce como la precisión de la máquina y se llama ϵ_m . Puede chequear de manera simple en consola, abra un IPython y ejecute

```
1 In [1]: import sys
2
3 In [2]: sys.float_info.epsilon
4
5 Out[2]: 2.220446049250313e-16
```

Su salida debe ser algo como $2.220446049250313e-16$. Eso significa que la diferencia mínima para distinguir entre dos números flotantes es $\sim 10^{-16}$. Por esa razón, al comparar dos números flotantes se debe tener cuidado.

```
1 In [26]: a = 0.1
2
3 In [27]: b = 0.2
4
5 In [28]: a+ b
6 Out[28]: 0.30000000000000004
7
8 In [29]: a+b == 0.3
9 Out[29]: False
10
11 tolerancia = 1e-5
12
13 In [33]: abs(a+b - 0.3) <= tolerancia
14 Out[33]: True
```

Estos errores de redondeo ocurren porque la máquina tiene una memoria finita y los números reales son infinitos. Si operamos muchas veces con los números redondeados, se propagará el error.

Problemas:

1. Modifique el programa `tiempo_vuelo.py` para que pida el nombre de un planeta en el sistema solar, calcule el tiempo de caída en ese planeta y lo compare con la Tierra.
2. Construya un programa usando la función `range` que pida un número entero e imprima el cuadrado y el cubo de la lista en ese range. Ejemplo si el número es 3, entonces debe imprimir cuadrado = 0, 1, 4 y cubo = 0, 1, 8.
3. **Sumatoria discreta y la constante de Stefan-Boltzmann:** Escriba un programa que solicite al usuario un número entero N y calcule la siguiente sumatoria utilizando un bucle `for`:

$$S_N = \sum_{k=1}^N \frac{1}{k^4}$$

Compare el resultado obtenido para $N = 10, 100, 1000$ con el valor exacto del límite infinito $\sum_{k=1}^{\infty} \frac{1}{k^4} = \frac{\pi^4}{90}$.

4. **Producto escalar y trabajo mecánico:** El trabajo realizado por una fuerza $\vec{F}$ a lo largo de un desplazamiento $\vec{r}$ se define como $W = \vec{F} \cdot \vec{r} = \sum_{i=1}^3 F_i r_i$. Dadas las listas:

```
1 F = [12.5, -3.2, 5.0] # Fuerza en N
2 r = [2.0, 4.5, -1.0] # Desplazamiento en m
```

Construya un programa que utilice un bucle **for** para calcular el producto escalar e imprima el trabajo total en Joules.

5. **Fórmula de Rydberg y Series Espectrales del Hidrógeno:** En 1888, Johannes Rydberg publicó su famosa fórmula para determinar las longitudes de onda λ de las líneas de emisión del átomo de hidrógeno:

$$\frac{1}{\lambda} = R \left(\frac{1}{m^2} - \frac{1}{n^2} \right)$$

donde $R = 1,097 \times 10^{-2} \text{ nm}^{-1}$ es la constante de Rydberg y m, n son números enteros positivos con $n > m$. Para un valor fijo de m , los distintos valores de n generan una serie espectral (las tres primeras, $m = 1, 2, 3$, corresponden a las series de Lyman, Balmer y Paschen, respectivamente).

Construya un programa, utilizando bucles **for** anidados, calcule e imprima en pantalla la longitud de onda λ (en nanómetros) para las primeras 5 líneas ($n = m + 1, \dots, m + 5$) de cada una de las tres primeras series ($m = 1, 2, 3$).

try - except

Ya hemos visto que que en los programas pueden aparecer errores cuando los ejecutamos. Por jemplo, dividir por cero, calcular raíces negativas en los reales, sumar una cadena con un entero. Normalmente, hemos usado el **if** para ocntrolar dichos errores, sin embargo, este es un poco pesado.

Hay una estructura de control dedicada a la detección y el tratamiento de excepciones: **try-except**. Su sintaxis básica es la siguiente

```
1 try:
2     # acción con posible error
3 except:
4     # acción para tratar el error
```

La idea es algo como esto: «intenta ejecutar estas acciones, si hay un error, entonces ejecuta esto otro »

Podemos ver esto con un ejemplo simple. Suponga que queremos mostrar la solución de $ax + b = 0 \implies x = -b/a$

try_except.py

```
1 """Uso básico del try-except"""
2 a = float(input("Entre a\n"))
3 b = float(input("Entre b\n"))
4
5 solution = -b/a
6
7 print(f"La soución es x = {solution: .3f}")
```

Al ejecutar el programa, si se ingresa la variable $a = 0$, sale el error `ZeroDivisionError: division by zero`. Lo podemos resolver así:

try_except_v2.py

```
1 """Uso básico del try-except"""
2 try:
3     a = float(input("Entre a\n"))
4     b = float(input("Entre b\n"))
5
6     solution = -b / a
7
8 except ZeroDivisionError: # analiza si a =0
9     print("Error: El coeficiente 'a' debe ser diferente de cero.")
10
11 except ValueError: # analiza si alguna variable que entra no es un número
12     print("Error: Debe ingresar un valor numérico válido.")
13
14 else:
15     # Este bloque solo se ejecuta si la división en try fue exitosa
16     print("Las variables ingresadas son correctas\n")
17     print(f"La solución es x = {solution:.3f}")
```

Si todas las instrucciones en el interior del `try` se ejecutan correctamente, el programa salta hasta el `else` y omite el resto de código. Por otro lado, si ocurre un error, el programa se interrumpe y busca en el bloque `except` el error que coincida con dicha falla. En nuestro ejemplo esto puede pasar por dos cosas, una división por cero o que no se ingrese un número.

Conceptos clave del manejo de excepciones

- **try:** Delimita las instrucciones riesgosas (lectura de datos y operaciones algebraicas).
- **except:** Intercepta errores específicos (`ZeroDivisionError`, `ValueError`) y ejecuta un protocolo de recuperación sin detener el programa.
- **else:** Bloque opcional que se ejecuta únicamente cuando el `try` concluye sin ningún error.

Ejemplo: Para finalizar esta parte, considere el siguiente ejemplo

ecuacion_segundo_grado.py

```
1 """Uso básico del try-except"""
2 import math
3
4 try:
5     a = float(input("Entre a\n"))
6     b = float(input("Entre b\n"))
7     c = float(input("Entre c\n"))
8
9     d = b**2 - 4 * a * c #discriminante
10
11     x1 = (-b - math.sqrt(d))/(2 * a)
```

```
12     x2 = (-b + math.sqrt(d))/(2 * a)
13
14 except ZeroDivisionError: # analiza si a =0
15     print("Error: El coeficiente 'a' debe ser diferente de cero.")
16
17 except ValueError: # analiza si alguna variable que entra no es un número
18     print("Error: Puede que no haya ingresado un valor numérico\n o que no
19     haya solución en los reales.")
20
21 else:
22     # Este bloque solo se ejecuta si la división en try fue exitosa
23     print("Las variables ingresadas son correctas\n")
24     print(f"Las soluciones son x_1 = {x1:.3f} y x_2 = {x2:.3f} ")
```

Observación

Note cómo `math.sqrt(d)` lanza un `ValueError` cuando $d < 0$. Si quisiéramos admitir soluciones complejas sin caer en la excepción, deberíamos emplear la librería `cmath` en lugar de `math`.